EJAVA UNIT-2

1. Inheritance

- Inheritance is a mechanism in which one object acquires all the properties and behaviours of a parent object.
- The idea behind inheritance is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also.
- Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

Why use inheritance:

➢ For Method Overriding (so runtime polymorphism can be achieved).
➢ For Code Reusability.

2. Terms used in Inheritance:

- **Class** → Blueprint for creating objects.
- **Subclass / Child class** → Class that inherits another class.
- **Superclass / Parent class** → Class whose properties are inherited.
- **Reusability** → Reusing existing methods and variables in a new class.

3. Syntax and Example

Syntax:

```
class ChildClass extends ParentClass{
   // methods and fields
}
```

Example:

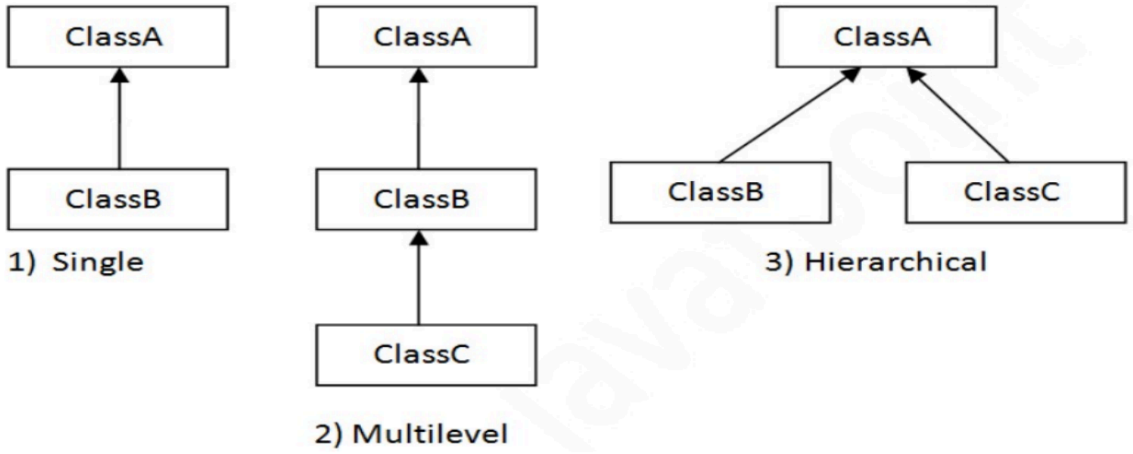
```
class Employee{
   float salary = 40000;
}

class Programmer extends Employee{
   int bonus = 10000;

   public static void main(String args[]){
      Programmer p = new Programmer();

      System.out.println(p.salary);
      System.out.println(p.bonus);
   }
```

}

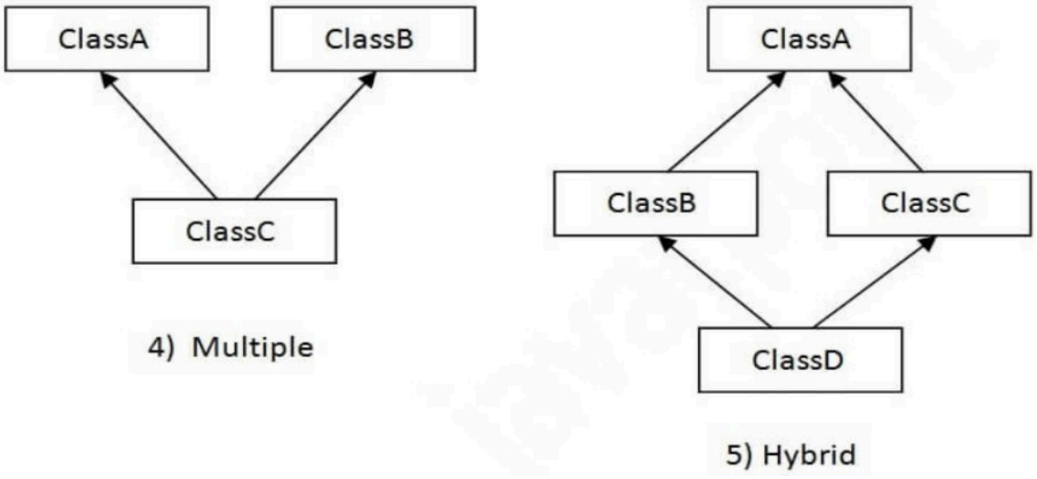
Output:

40000
10000

4. Types of inheritance



Note: Multiple and Hybrid inheritance is supported through interface only



Types of Inheritance:

1. **Single Inheritance** → One child class inherits one parent class.
2. **Multilevel Inheritance** → A class inherits another class, and that class further inherits another class.
3. **Hierarchical Inheritance** → Multiple child classes inherit a single parent class.

4. **Multiple Inheritance** → One child class inherits multiple parent classes. *(Not supported directly in Java)*
5. **Hybrid Inheritance** → Combination of two or more inheritance types. *(Supported through interfaces only)*

5. Why multiple inheritance is not supported in Java?

Multiple inheritance is not supported in Java **to reduce complexity and avoid ambiguity problems** (diamond problem), making the language simpler and easier to manage.

6. Super Keyword

- The super keyword in Java is a reference variable which is used to refer immediate parent class object.
- Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of Java super Keyword:
➢ super can be used to refer immediate parent class instance variable.
➢ super can be used to invoke immediate parent class method.
➢ super() can be used to invoke immediate parent class constructor.

```
class Animal {
  String color = "White";

  Animal() {        // Parent constructor
    System.out.println("Parent constructor called");
  }

  void sound() {     // Parent method
    System.out.println("Animal makes sound");
  }
}

class Dog extends Animal {

  String color = "Black";

  Dog() {
    super(); // 3) Invokes parent class constructor

    System.out.println(super.color); // 1) Refers parent class variable
```

```
    super.sound(); // 2) Invokes parent class method
  }

  public static void main(String args[]) {
    Dog d = new Dog();
  }
}
```

7. Method Overriding

If a child class provides its own implementation of a method that is already defined in the parent class with the **same method name and same parameters**, it is called **method overriding**. It allows the child class to change the behavior of the parent class method.

Uses of Method Overriding:

- Used to provide a **specific implementation** of a method already present in parent class.
- Used to achieve **Runtime Polymorphism**.

Example:

```
class Animal{
  void sound(){
    System.out.println("Animal makes sound");
  }
}

class Dog extends Animal{

  // Overriding parent method
  void sound(){
    System.out.println("Dog barks");
  }
}

class Main{
  public static void main(String args[]){

    Dog d = new Dog();

    d.sound();
  }
```

}

Output:

Dog barks

Points to remember:

- Method name must be same
- Parameters must be same
- Inheritance is required
- Used for runtime polymorphism
- Static methods cannot be overridden

Here sound() exists in both parent and child classes, but the child class implementation is executed.

8. Differences between Method overloading and Method overriding

No	Method Overloading	Method Overriding
1	Method overloading is used to **increase readability** of the program.	Method overriding is used to **provide specific implementation** of a method already provided by parent class.
2	Method overloading is performed **within the same class**.	Method overriding occurs in **two classes with IS-A relationship (inheritance)**.
3	In method overloading, **parameters must be different**.	In method overriding, **parameters must be same**.
4	Method overloading is an example of **compile-time polymorphism**.	Method overriding is an example of **runtime polymorphism**.
5	Overloading **cannot be done by changing return type alone**. Return type may be same or different, but parameters must change.	Return type must be **same or covariant**.
6	Inheritance is **not required**.	Inheritance is **required**.

7	Methods can be **static**.	Static methods **cannot be overridden** (they are hidden).
8	Method call is resolved by the **compiler**.	Method call is resolved by the **JVM at runtime**.

9. Runtime Polymorphism / Dynamic Method Dispatch

Runtime Polymorphism / Dynamic Method Dispatch

Runtime polymorphism is a process in which a call to an **overridden method is resolved at runtime instead of compile time**. Here, the method that executes depends on the **object referred by the reference variable**, not the reference type.

Points:

- Achieved using **method overriding**
- Method is decided by **JVM at runtime**
- Parent reference can refer to child object (**upcasting**)

Example:

```
class A{
  void show(){
    System.out.println("Class A");
  }
}

class B extends A{
  void show(){
    System.out.println("Class B");
  }
}

class Main{
  public static void main(String args[]){

    A obj = new B(); // upcasting

    obj.show();
  }
}
```

Output:

Class B

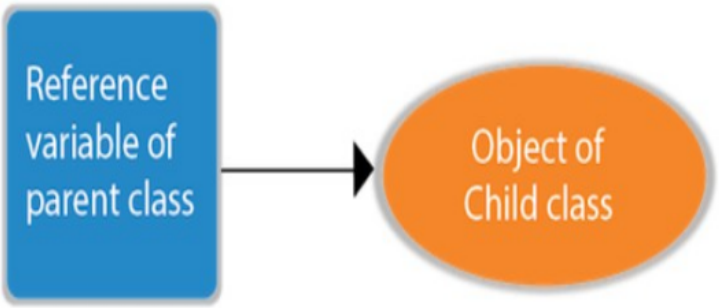
Here obj is of type A, but it refers to object B, so overridden method of B executes.

10. Upcasting

Upcasting is the process of **referring a child class object using a parent class reference variable**. It is used in runtime polymorphism.

Syntax:

Parent obj = new Child();



Example:

```
class A{
}

class B extends A{
}

class Main{
  public static void main(String args[]){

    A obj = new B(); // Upcasting
  }
}
```

Points:

- Parent reference points to child object
- Used for runtime polymorphism
- Supports **IS-A relationship**

11. Runtime Polymorphism with Data Member

Runtime polymorphism **cannot be achieved using data members (variables)** because **only methods are overridden, not variables**. Variable access depends on the **reference type**, not the object type.

Example:

```
class Bike{
  int speed = 90;
}

class Honda extends Bike{
  int speed = 150;
}

class Main{
  public static void main(String args[]){

    Bike obj = new Honda();

    System.out.println(obj.speed);
  }
}
```

Output:

90

Here obj refers to Honda, but output is 90 because variables are resolved using the **reference type (Bike)**.

12. Abstraction

Abstraction is the process of **hiding implementation details and showing only essential functionality to the user**. It allows users to focus on **what an object does instead of how it works**.

Example: Sending SMS → you type and send the message, but you do not know the internal processing.

Ways to achieve abstraction in Java:

1. **Abstract Class** *(0–100% abstraction)*
2. **Interface** *(100% abstraction)*

13. Abstract Class

A class declared using the abstract keyword is called an **abstract class**. It can contain **abstract methods** (without body) and **normal methods** (with body). An abstract class cannot be instantiated (object cannot be created).

Points:

- Declared using abstract keyword
- Can have abstract and normal methods
- Can have constructors, variables, and methods
- Object cannot be created

Example:

```
abstract class Animal{

  abstract void sound(); // abstract method
}

class Dog extends Animal{

  void sound(){
    System.out.println("Dog barks");
  }
}

class Main{
  public static void main(String args[]){

    Dog d = new Dog();
    d.sound();
  }
}
```

Output:

```
Dog barks
```

14. Abstract class having constructor, data member and methods

An abstract class can contain **constructors, data members, abstract methods, and normal methods**. Although objects cannot be created for an abstract class, its constructor is called when a child object is created.

Example:

```
abstract class Bike{
```

```
  int speed = 100;   // data member

  Bike(){          // constructor
    System.out.println("Bike constructor called");
  }

  abstract void run(); // abstract method

  void display(){     // normal method
    System.out.println("Speed = " + speed);
  }
}

class Honda extends Bike{

  void run(){
    System.out.println("Bike is running");
  }
}

class Main{
  public static void main(String args[]){

    Honda h = new Honda();

    h.run();
    h.display();
  }
}
```

Output:

```
Bike constructor called
Bike is running
Speed = 100
```

15. Interface

An interface in Java is a **blueprint of a class**. It is used to achieve **abstraction** and **multiple inheritance** in Java. It contains **abstract methods and static constants** and cannot be instantiated.

Points:

- Used for **100% abstraction**
- Supports **multiple inheritance**
- Represents **IS-A relationship**
- Object cannot be created
- Methods are public abstract by default
- Variables are public static final by default

Example:

```
interface Animal{
  void sound();
}

class Dog implements Animal{

  public void sound(){
    System.out.println("Dog barks");
  }
}

class Main{
  public static void main(String args[]){

    Dog d = new Dog();
    d.sound();
  }
}
```

Output:

```
Dog barks
```

16. How to declare an interface?

Syntax:

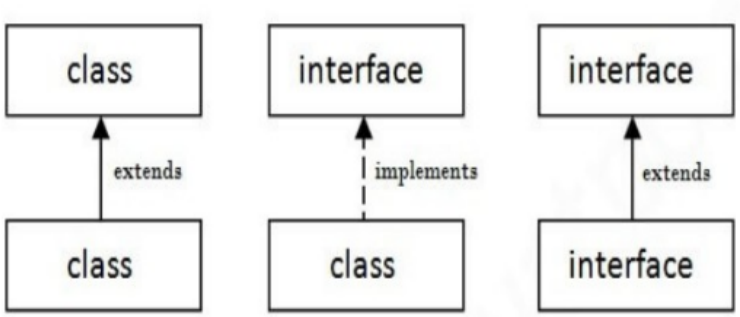
```
interface <interface_name>
{
  // declare constant fields
  // declare methods that abstract
  // by default.
}
```

17. Relationship between classes and interfaces

- A **class extends another class**
- A **class implements an interface**
- An **interface extends another interface**
- A class can implement **multiple interfaces**
- Interface supports **multiple inheritance**

Syntax:

class B extends A implements I1,I2



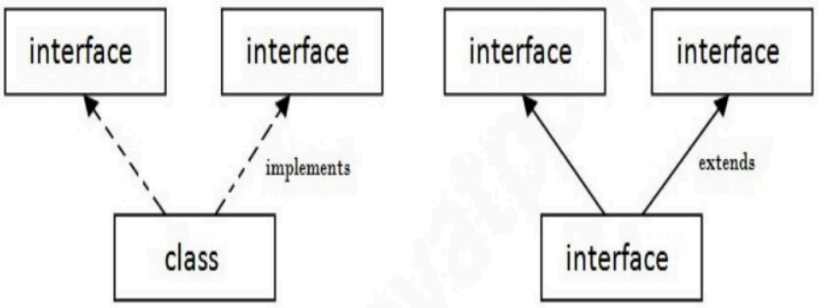
Here:

- B IS-A A
- B IS-A I1
- B IS-A I2

18. Multiple Inheritance by Interface

If a class **implements multiple interfaces**, or an interface **extends multiple interfaces**, it is called **multiple inheritance by interface**. Java supports multiple inheritance through interfaces.

Example:




```
interface A{
  void show();
}

interface B{
  void display();
}

class Test implements A,B{

  public void show(){
    System.out.println("Show method");
  }

  public void display(){
    System.out.println("Display method");
```

```
  }
}

class Main{
  public static void main(String args[]){

    Test t = new Test();

    t.show();
    t.display();
  }
}
```

Output:

```
Show method
Display method
```

Got it. I'll keep code minimal from now.

19. Interface Inheritance

An **interface can inherit another interface** using the extends keyword. The child interface gets the methods of the parent interface.

Example:

```
interface A{
  void show();
}

interface B extends A{
}

class Test implements B{
  public void show(){
    System.out.println("Hello");
  }
}

class Main{
  public static void main(String args[]){
    new Test().show();
```

```
  }
}
```

Output:

Hello

20.Differences between Abstract class and Interface

No	Abstract Class	Interface
1	Abstract class can have **abstract and non-abstract methods**.	Interface can have **only abstract methods**. Since Java 8, it can also have **default and static methods**.
2	Abstract class **doesn't support multiple inheritance**.	Interface **supports multiple inheritance**.
3	Abstract class can have **final, non-final, static and non-static variables**.	Interface has only **static and final variables**.
4	Abstract class **can provide implementation** of interface.	Interface **can't provide implementation of abstract class**.
5	abstract keyword is used to declare abstract class.	interface keyword is used to declare interface.
6	Abstract class can **extend another class** and **implement multiple interfaces**.	Interface can **extend another interface only**.
7	Abstract class is inherited using **extends** keyword.	Interface is implemented using **implements** keyword.
8	Abstract class members can have access modifiers like **private, protected, public**, etc.	Members of interface are **public by default**.

21. Final Keyword

The final keyword is used to **restrict the user**. It can be used with **variables, methods, and classes**.

Uses:

1. **Final Variable** → Value cannot be changed (constant)

```
final int x = 10;
```

2. **Final Method** → Method cannot be overridden

```
final void show(){}
```

3. **Final Class** → Class cannot be inherited

```
final class A{}
```

Points:

- Final variable → value fixed
- Final method → cannot override
- Final class → cannot extend

22. Aggregation

Aggregation is a relationship where **one class contains a reference of another class**. It represents a **HAS-A relationship** and is used for **code reusability**.

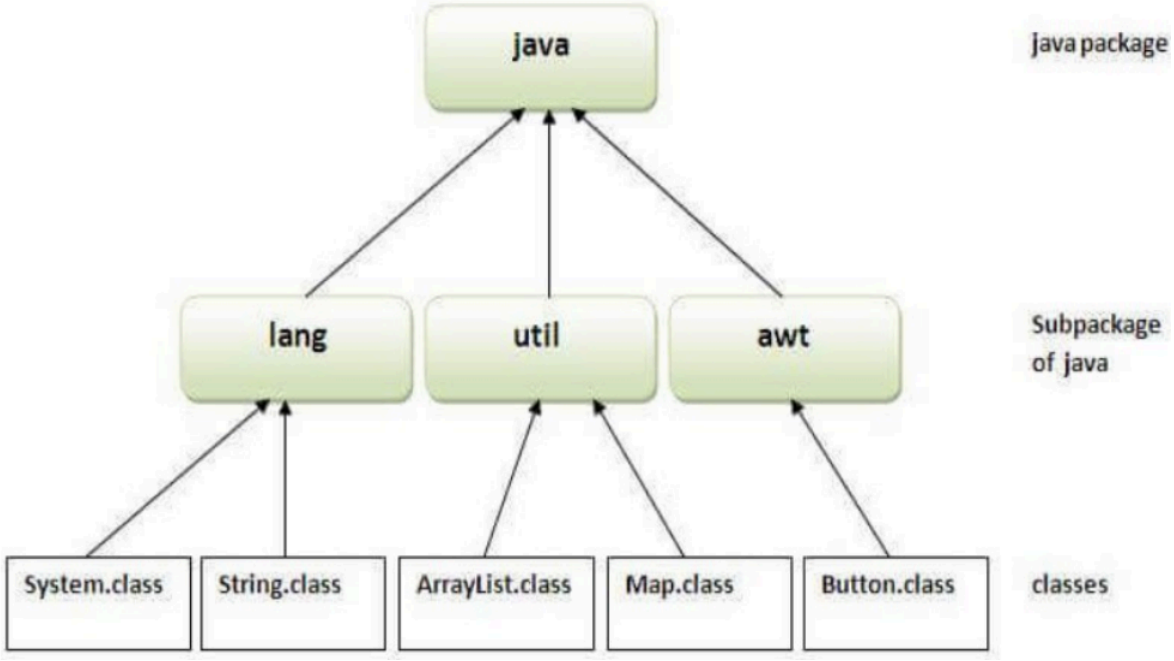
Example:

```
class Address{
  String city = "Bangalore";
}

class Student{
  Address a = new Address(); // HAS-A relationship
}

class Main{
  public static void main(String args[]){
    Student s = new Student();
    System.out.println(s.a.city);
  }
}
```

Output:

Bangalore

23. Packages



A package in Java is a **group of similar classes, interfaces, and sub-packages**. It is used to **organize classes and avoid naming conflicts**.

Advantages:

- Avoid naming conflicts
- Provides access protection
- Makes programs easy to maintain

Syntax:

```
package mypack;
```

Example:

```
package mypack;

class Test{
  public static void main(String args[]){
    System.out.println("Hello");
  }
}
```

24. Simple example of creating Java package

```
package mypack;

public class Test{
  public static void main(String args[]){
    System.out.println("Package Created");
  }
}
```

Compile:
javac -d . Test.java

Run:
java mypack.Test

Output:
Package Created

25. How to access package from another package?

There are **3 ways**:

1. **Using packagename.***
   Imports all classes of the package.

```
import mypack.*;
```

2. **Using packagename.classname**
   Imports only a specific class.

```
import mypack.Test;
```

3. **Using fully qualified name**
   No need of import.

```
mypack.Test t = new mypack.Test();
```

26. Sub-package

A package inside another package is called a **sub-package**. It is used to further organize and categorize classes.

Example:

```
package mypack.testpack;

class Test{
   public static void main(String args[]){
      System.out.println("Sub Package");
   }
}
```

Import sub-package:

```
import mypack.testpack.*;
```

or

```
import mypack.testpack.Test;
```

27. Package – Interface Example

Interface can be created inside a package and then implemented in another class.

Interface inside package:

```
package mypack;

public interface Show{
   void display();
}
```

Implementing interface:

```
import mypack.Show;

class Test implements Show{

   public void display(){
```

```
    System.out.println("Hello");
  }

  public static void main(String args[]){
    new Test().display();
  }
}
```

Output:

Hello

28. Exception Handling

Exception handling is a mechanism used to **handle runtime errors** and maintain the **normal flow of the program**. An exception is an **abnormal condition or event that disrupts program execution**.

Why use exception handling?

- Prevents program termination
- Maintains normal flow of execution
- Allows remaining statements to execute

Example:

Without exception handling:

```
int a = 10/0;
System.out.println("Hello");
```

Program stops due to exception.

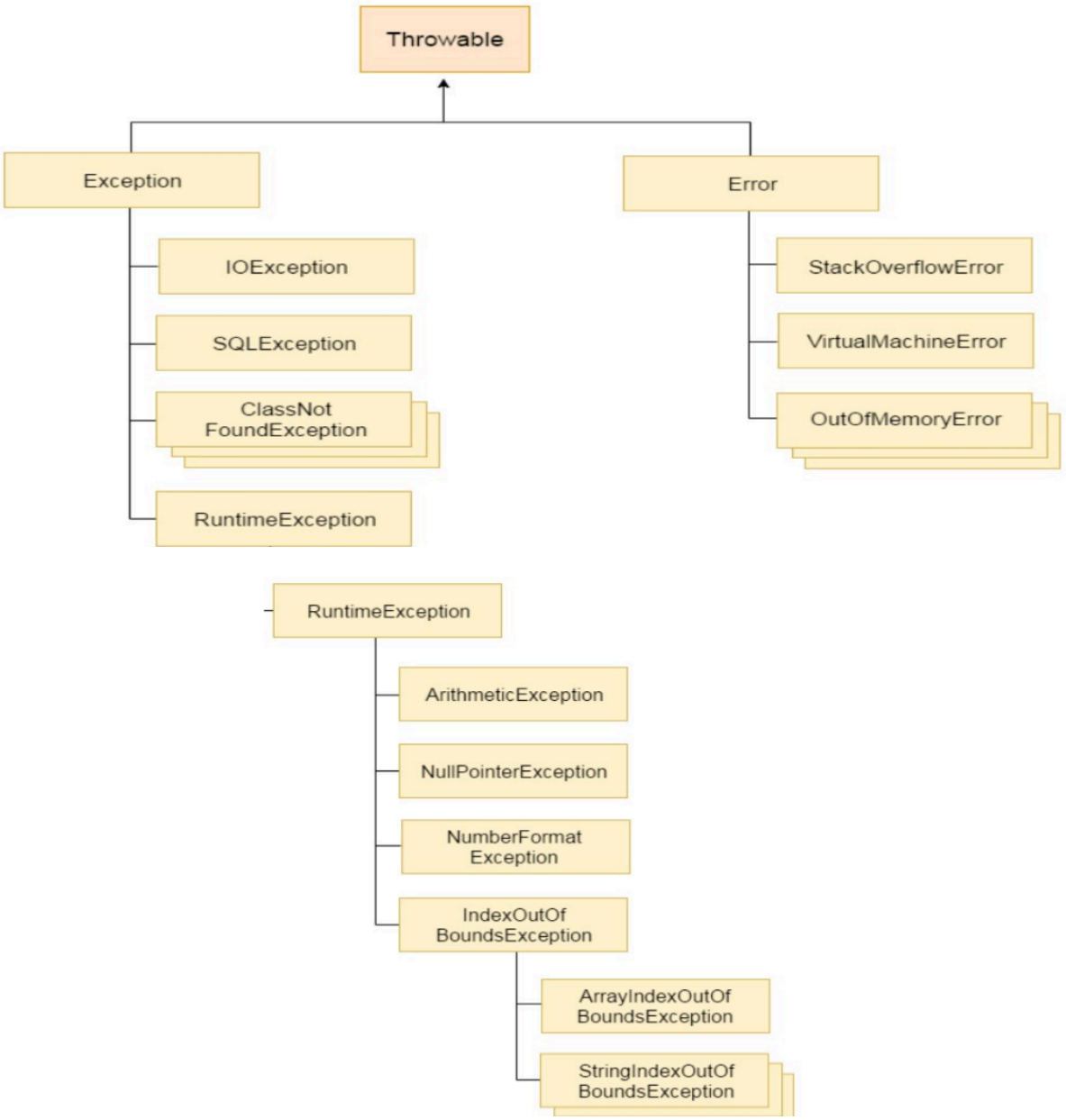
With exception handling:

```
try{
  int a = 10/0;
}
catch(Exception e){
  System.out.println("Error");
}

System.out.println("Hello");
```

Output:

Error
Hello

29.Hierarchy of Java Exception classes



- **Error** → Irrecoverable errors
- **Exception** → Can be handled
- **RuntimeException** → Unchecked exceptions

30. Types of Java Exceptions

1. **Checked Exception**
    - Checked at **compile time**
    - Must be handled using try-catch or throws
    - Examples: IOException, SQLException
2. **Unchecked Exception**
    - Checked at **runtime**
    - Not checked at compile time
    - Examples: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException
3. **Error**
    - Irrecoverable condition
    - Cannot normally be handled
    - Examples: OutOfMemoryError, VirtualMachineError

31. Common Scenarios of Java Exceptions

1. **ArithmeticException** → Divide by zero

```
int a = 50/0;
```

2. **NullPointerException** → Accessing method using null object

```
String s = null;
s.length();
```

3. **NumberFormatException** → Invalid conversion of string to number

```
String s="abc";
int i=Integer.parseInt(s);
```

4. **ArrayIndexOutOfBoundsException** → Accessing invalid array index

```
int a[]=new int[5];
a[10]=50;
```

32. Java Exception Keywords

There are **5 keywords** used in exception handling:

- **try** → Contains code that may generate exception
- **catch** → Handles the exception
- **finally** → Executes whether exception occurs or not
- **throw** → Used to explicitly throw an exception
- **throws** → Declares exceptions that may occur in a method

33. Java try-catch block

The try block contains code that may generate an exception, and the catch block is used to handle that exception. If an exception occurs inside try, the remaining statements in that block are skipped and control moves to catch.

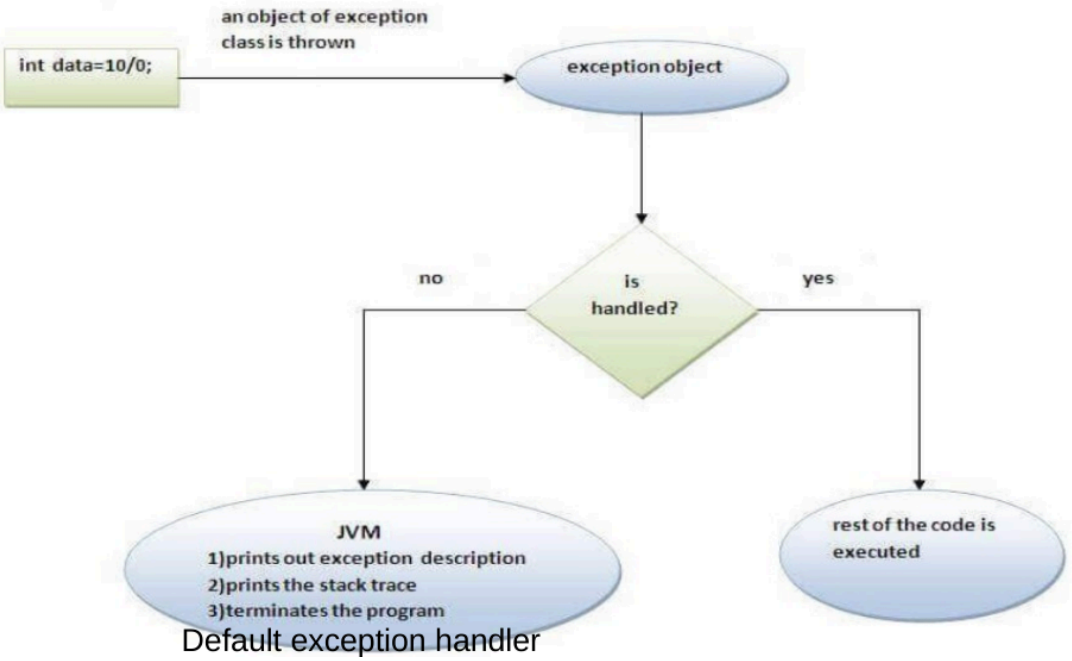
Example:

```
try{
   int a = 10/0;
}
catch(Exception e){
   System.out.println("Exception handled");
}
```

Output:

Exception handled

34. Internal working of java try-catch block



Default exception handler

35. Java catch multiple exceptions

A try block can be followed by **multiple catch blocks**. Each catch block handles a **different type of exception**. When an exception occurs, Java executes only the matching catch block.

Example:

```
try{
   int a[] = new int[5];
   a[10] = 50;
}
catch(ArithmeticException e){
   System.out.println("Arithmetic Error");
}
catch(ArrayIndexOutOfBoundsException e){
   System.out.println("Array Error");
}
catch(Exception e){
   System.out.println("Other Exception");
}
```

Output:

Array Error

36. Java Nested try block

A try block inside another try block is called a **nested try block**. It is used when one part of code may generate a different exception inside another exception-handling block.

Example:

```
try{

   try{
      int a = 10/0;
   }
   catch(ArithmeticException e){
      System.out.println("Arithmetic Error");
   }
```

Syntax:

```
....
try
{
    statement 1;
    statement 2;
    try
    {
        statement 1;
        statement 2;
    }
    catch(Exception e)
    {
    }
}
catch(Exception e)
{
}
....
```

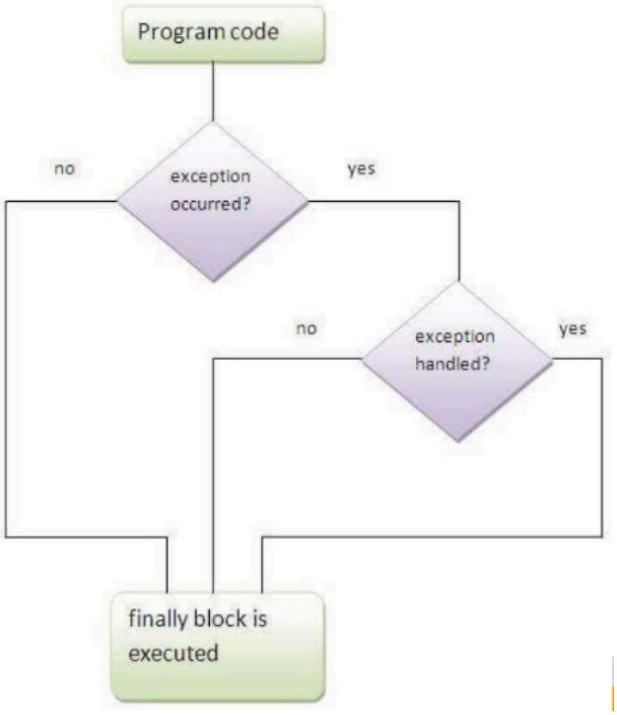
```
}
catch(Exception e){
   System.out.println("Exception");
}
```

Output:

Arithmetic Error

37. Java finally block

The finally block is used to execute important code such as **closing files or releasing resources**. It executes **whether an exception occurs or not**.



Example:

```
try{
   int a = 10/0;
}
catch(Exception e){
   System.out.println("Exception");
}
finally{
   System.out.println("Finally block");
}
```

Output:

Exception
Finally block

38. Different cases where Java finally block can be used

Case 1: Exception **does not occur** → finally executes

```
try{
  System.out.println("Try");
}
finally{
```

```
  System.out.println("Finally");
}
```

Output:

```
Try
Finally
```

Case 2: Exception **occurs but not handled** → finally executes before program termination

```
try{
  int a=10/0;
}
finally{
  System.out.println("Finally");
}
```

Case 3: Exception **occurs and handled** → finally executes

```
try{
  int a=10/0;
}
catch(Exception e){
  System.out.println("Exception");
}
finally{
  System.out.println("Finally");
}
```

Output:

```
Exception
Finally
```

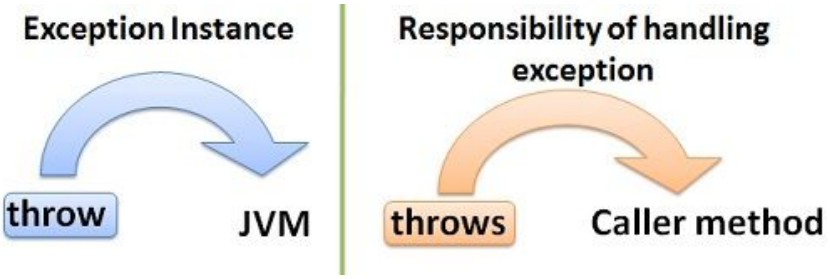
finally is always executed whether exception is handled or not. It is commonly used for cleanup operations like closing files or database connections.

39. Java throw Keyword

The throw keyword is used to **explicitly throw an exception**. It transfers control to the nearest catch block. It is mainly used to create **custom exceptions or manually generate exceptions**.

Syntax:

throw new ExceptionType();



Example:

```
int age = 15;

if(age < 18)
   throw new ArithmeticException("Not eligible");
```

Output:

```
Exception in thread "main"
java.lang.ArithmeticException: Not eligible
```

40. Java throws Keyword

The throws keyword is used to **declare exceptions that may occur in a method**. It informs the programmer that a method can throw an exception, so proper exception handling can be done. It is mainly used for **checked exceptions**.

Syntax:

```
returnType method() throws ExceptionName{
}
```

Example:

```
class Test{

   static void check() throws Exception{
      throw new Exception();
   }

   public static void main(String args[]) throws Exception{
```

```
        check();
    }
}
```

Difference:

- throw → used to throw an exception
- throws → used to declare an exception

41. Differences between throw and throws

No	throw	throws
1	throw keyword is used to **explicitly throw an exception**.	throws keyword is used to **declare an exception**.
2	Checked exceptions **cannot be propagated using throw alone**.	Checked exceptions **can be propagated using throws**.
3	throw is followed by an **instance/object**.	throws is followed by an **exception class name**.
4	throw is used **inside the method body**.	throws is used with the **method signature**.
5	Cannot throw **multiple exceptions at once**.	Can declare **multiple exceptions**.

Example:

```
throw new ArithmeticException();
void test() throws IOException, SQLException
```

42. Java Custom Exception

A custom exception is a **user-defined exception** created by extending the Exception class. It is used when predefined exceptions are not suitable for application requirements.

Example:

```
class InvalidAgeException extends Exception{
}
```

```
class Test{
   public static void main(String args[]) throws Exception{

      int age = 15;

      if(age < 18)
         throw new InvalidAgeException();
   }
}
```

Here InvalidAgeException is a custom exception created by the programmer.